Adversarial SQLi Detection Using Character-Level CNN and Reinforcement Learning

Fucai Yu, *Member, IEEE*, Xusheng Li, Yang Wu, Ziqiang Chang, Gaolei Fei, *Member, IEEE*, and Xuemeng Zhai, *Member, IEEE*

***Abstract*—Adversarial SQL injection (SQLi) refers to the process in which attackers dynamically modify their attack strategies based on feedback from the target Web Application Firewall (WAF) in an attempt to bypass it. The emergence of automated adversarial SQLi tools in recent years, such as AdvSQLi, RAT, and GPTFuzzer, demonstrates that adversarial SQLi has become a critical method for vulnerability detection and SQLi execution. These tools' intelligent nature poses a significant threat to existing SQLi defense mechanisms. Payloads subjected to adversarial mutation can effectively deceive machine learning-based WAFs, while rule-based WAFs are more susceptible to having vulnerabilities discovered. To address this issue, we propose a hybrid model based on enhanced Character-Level CNN (CLCNN) and Reinforcement Learning (RL), which detects malicious patterns through multi-scale feature extraction by CLCNN from a pattern-matching perspective. Additionally, the reinforcement learning module is employed for adversarial training, executing de-obfuscation operations on payloads for which the CLCNN outputs a low confidence level, further processing ambiguous payloads. The experimental results indicate that even without a priori knowledge, CLCNN demonstrates strong resistance to such tools. And the RL module can further enhance Recall through adversarial training, with minimal reduction in the hybrid model's performance on standard SQLi datasets.**

***Index Terms*—Adversarial SQLi, Character-Level CNN, Reinforcement Learning, Web Application Firewall.**

I. Introduction

WITH the digital age's rapid advancement, countless services from financial systems and social media to enterprise platforms and smart homes have migrated to the cloud, storing massive volumes of sensitive data. SQL injection (SQLi) remains one of the most prevalent and damaging attack vectors. It embeds malicious SQL statements into web queries, enabling unauthorized access to server databases, data breaches, and system disruptions with severe consequences. To counter this threat, Web Application Firewalls (WAFs) act as a critical line of defense. Operating as reverse proxies, they filter malicious requests before they reach servers. However, WAFs are not invulnerable. Attackers can analyze their rules to bypass detection, as demonstrated by the 2023 ResumeLooters attack. Attackers exploited unpatched SQLi vulnerabilities in recruitment platforms to evade basic WAF protections, stealing over 10 million job seekers' personal identifiable information (PII) and highlighting the persistent risk of SQLi even with defensive measures in place. Historically, such targeted SQLi attacks required skilled hackers and extensive preparation, but recent advances have enabled automated adversarial SQLi tools. While designed to test WAF vulnerabilities and improve security, these tools are easily misused by cybercriminals. This misuse parallels the abuse of SQLMap, but with two key enhancements:

*1) **Expanded Malicious Payloads**:* Inspired by Adversarial Machine Learning (AML), researchers have focused on generating adversarial samples that induce mispredictions in detection models. For instance, Appelt et al. [1] proposed a Context-Free Grammar (CFG) to represent SQLi statement structures as parse trees, enabling modular generation of syntactically correct SQLi statements and their variants. Liang et al. [2] trained large language models on a SQLi payload corpus and leveraged reinforcement learning to generate effective adversarial payloads. Qu et al. [3] adopted CFG to parse existing SQLi payloads and perform adversarial mutations—these mutations retain the original malicious functionalities while modifying payload characteristics to evade target WAFs. Notably, Qu et al. specifically validated the mutated payloads on real-world commercial WAF-as-a-Service solutions, demonstrating robust bypass capabilities against mainstream enterprise-grade defenses.

*2) **Enhanced Search Efficiency**:* Drawing inspiration from Adaptive Random Testing (ART), several researchers have developed methods to adaptively select test cases with high potential to bypass target WAFs. For instance, Zhang et al. [4] defined a keyword frequency-based distance function, prioritizing payloads that are distant from known invalid cases to enhance bypass probability. Amouei et al. [5] first clustered payloads by similarity and then employed reinforcement learning to concentrate the search on clusters with proven effectiveness, improving the efficiency of identifying bypass-capable payloads.

In recent years, with the growing attention to and advancement of related research, more scholars are expected to focus on balancing testing space expansion and search efficiency improvement to achieve more effective SQLi vulnerability detection (maximizing success rates under constrained resources [6]). However, this progress also raises concerns: adversarial

Manuscript created November, 2024. (Corresponding author: Xuemeng Zhai.)

F. Yu, X. Li are with the School of Information and Communication Engineering, University of Electronic Science and Technology of China, 611731, Chengdu, China (e-mail: fcyu@uestc.edu.cn; 202321011119@std.uestc.edu.cn).

Y. Wu is with China Academy of Launch Vehicle Technology, 100076, Beingjing, China (e-mail: 905604274@qq.com).

Z. Chang is with Luoyang Institute of Electro-Optical Equipment, AVIC, 471009, Luoyang, China (e-mail: 1344784651@qq.com).

G. Fei and X. Zhai are with the School of Information and Communication Engineering, University of Electronic Science and Technology of China, 611731, Chengdu, China (e-mail: fgl@uestc.edu.cn; Zxm@uestc.edu.cn).

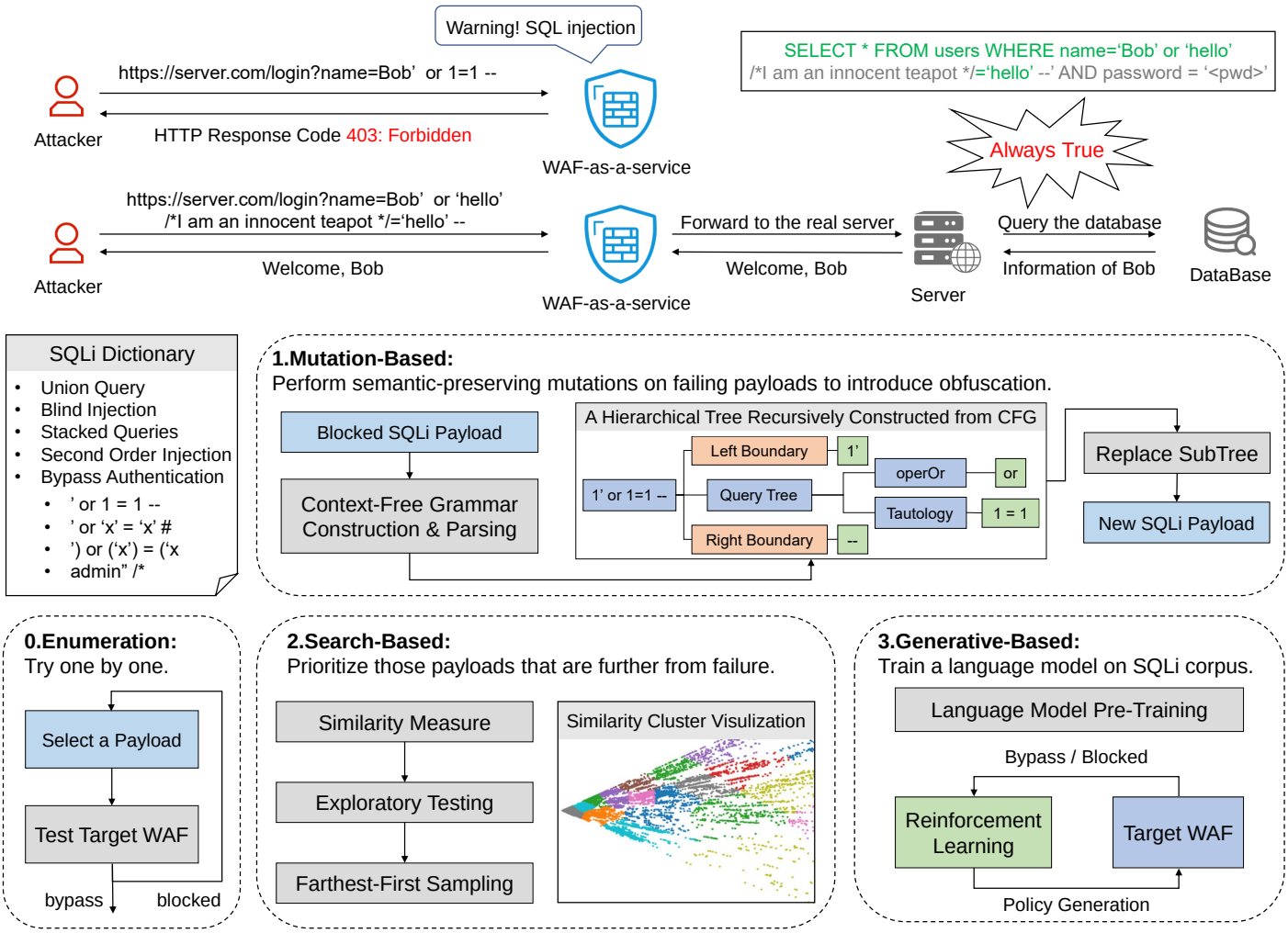


Fig. 1. Illustration of How Attackers Use SQLi Dictionaries to Conduct Attacks for User Privacy Data. Compared to traditional brute-force enumeration, mutation-based approaches obfuscate existing payloads while preserving semantics to bypass WAFs; search-based methods rapidly identify effective payloads in the dictionary; generation-based methods expand the dictionary to cover the test space comprehensively.

SQLi tools are likely to be widely abused for malicious purposes, inflicting substantial losses on unprepared service providers. Assessing the potential harms of such tools and implementing targeted countermeasures is therefore imperative.

To address this critical issue, we propose a hybrid model integrating an enhanced Character-Level CNN (CLCNN) and Reinforcement Learning (RL). This model combines the fine-grained, multi-scale feature extraction capabilities of CLCNN with the adversarial adaptation advantages of RL, achieving robust performance in both conventional and adversarial SQLi detection. The main contributions of this work are as follows:

1) We systematically summarize and validate the potential threats posed by automated adversarial SQLi tools, verifying their bypass performance on existing WAFs and identifying the inherent limitations of traditional ML-based detection methods.
2) To counter adversarial SQLi attacks, we propose a CLCNN+RL hybrid solution that exhibits strong resistance to such tools. After adversarial training, the model achieves higher Recall while minimally compromising performance in conventional detection scenarios.
3) The proposed RL module is designed as a plug-and-play component. Through customized training, it can enhance the adversarial resistance of other deployed machine learning models (without adjusting existing model parameters) and extend protection to other potential adversarial injection attacks (e.g., XSS).

II. Background and Releated Work

As illustrated in Fig. 1, adversarial SQLi attacks typically leverage dictionaries to launch targeted attempts, relying on three primary categories of attack tools. Compared to brute-force enumeration, mutation-based methods obfuscate existing payloads while preserving semantics for WAF bypass; search-based methods rapidly locate effective payloads in the dictionary; generation-based methods expand the test space via dictionary extension. Notably, authentication mechanisms can mitigate threats from such frequent-attempt attacks [7]–[9], but

TABLE I
EXAMPLES OF MUTATION METHODS PROPOSED BY [3], [10]–[12]

Mutation	Example
Case Swapping	or 1 = 1 → oR 1 = 1
Keywords Substitution	or 1 = 1 → || 1 = 1
Whitespace Substitution	or 1 = 1 → \tor1\n=1
Comment Injection	or 1 = 1 → /*xxx*/or 1 =/*hello*/1
Comment Rewriting	/*xxx*/or 1 = 1 → /*abc*/or 1 = 1
Integer Encoding	or 1 = 1 → or 0x1 = 1
Nested Query	or 1 = 1 → or 1 = (SELECT 1)
Logical Invariant	or 1 = 1 → or 1 = 1 and 'a' = 'a'
Operator Swapping	or 1 = 1 → or 1 like 1
Inline Comment	union select → /*!union*/ /*!50000select*/
Tautology Substitution	1 = 1 → (select ord('r') regexp 114) = 0x1
Recursive Encoding	or%201%3D1 → or%25201%253D1

as a general security measure applicable to various network attacks (e.g., traditional SQLi, brute-force attacks) rather than a solution specific to adversarial SQLi, it is not considered in this work.

A. Mutation-based Methods

Mutation-based methods are inspired by adversarial machine learning. They introduce semantically preserved perturbations into the payload to create adversarial samples that are misclassified by the victim WAF, thereby allowing the originally blocked payload to bypass the victim WAF while retaining the same malicious functionality, as shown in (1):

$$p^* = \arg\min_{p, C(p)} \mathcal{D}(f(p), c_t) \tag{1}$$

Where, p represents the original payload, p^* denotes the adversarial payload, f is the victim classifier, c_t is the classification target the adversary aims to achieve, $\mathcal{D}$ is the distance function, and $C(p)$ denotes the constraints that must not be violated during the search for adversarial examples.

In the SQLi mutation problem, researchers focus on selecting mutation strategies that minimize the number of interactions with the target WAF, enabling blocked payloads to bypass the WAF through semantically invariant mutations. Some commonly used mutation operators are shown in Table I, which primarily include synonym replacement of keywords, the insertion of obfuscation fields such as comments or placeholders, URL encoding, integer encoding, etc.

*1) **WAF-A-MoLE:*** Demetrio et al. [11] designed a priority queue to store mutated SQLi payloads along with their corresponding classification confidence scores (i.e., the probability that the target classifier predicts the payload as malicious). In each iteration, the algorithm selects the payload with the lowest confidence score from the queue for mutation. The newly mutated payload and its updated confidence score are reinserted into the priority queue. This process is repeated until a payload with sufficiently low confidence is found. Since WAF-A-MoLE always selects the most promising payload from all mutated payloads, the probability of bypassing the target WAF increases with the number of iterations. The algorithm is simple and easy to implement, but it degenerates into random mutations in real-world scenarios or black-box testing (in which the target classifier can only return the predicted label).

*2) **DRL:*** In order to achieve a more effective mutation strategy, Hemmati et al. [12] used reinforcement learning to select the most appropriate mutation operation in each state. They first used FastText to train an embedding model on the SQLi dataset to tokenize the payload and convert it into a fixed-length vector as the agent's observed state in the reinforcement learning model. They also used the mutation operators shown in Table I as the action space. They manually defined a set of reward values based on the HTTP response code, rewarding successfully bypassed payloads while punishing those that were blocked or erroneous. Ultimately, the model determined the appropriate action to take in each observed state. In addition, in order to encourage the model to take fewer actions, they additionally defined $R = R_t - \rho \times step$ to punish meaningless actions, where R_t is the reward obtained at the current time step, $step$ is the number of actions that have been performed in this episode, and ρ is a hyperparameter.

The problem with this approach is that for each target WAF, a large number of samples are required to train a reinforcement learning model, which will result in too many interactions with the target WAF and attract the attention of defenders.

*3) **AdvSQLi:*** To address the semantic errors that may be caused by the above two methods (such as mutating 'handle' to 'h&le'), Qu et al. [3] do not simply treat the payload as a string, but parse it into a hierarchical tree as shown in Fig. 1, where each leaf node is an atomic token in the SQL statement. Then, according to the context-free grammar, the subtree of the parsed tree is iteratively replaced with another equivalent form that retains the original malicious functionality. In constructing the hierarchical tree, they traverse the atomic tokens in the payload that can undergo mutation, while locking other constants to ensure that irrelevant components are retained in the final payload. Then, they employ the Monte Carlo Tree Search algorithm to identify the optimal mutation strategy, where each node in the search tree represents a hierarchical tree, and each edge corresponds to a mutation operation. The main steps of the algorithm are as follows:

1) **Selection:**Using the Upper Confidence Bounds (UCB) algorithm shown in equation (2) to select the most promising hierarchical tree for exploration.
2) **Expansion:**Randomly selecting an operable node within the tree and performing a random mutation to generate a new child node.
3) **Simulation:**Test whether the newly generated child node can evade the target WAF and get reward.
4) **Back-propagation:**Propagate the simulation result up the tree, updating the Q-values and visit counts of the parent nodes.

$$\text{score}_{ucb} = \arg\max_{v' \in \ children\ of\ v} \left(\frac{Q(v')}{N(v')} + c\sqrt{\frac{2\ln N(v)}{N(v')}}\right) \tag{2}$$

Where Q represents the cumulative reward, N denotes the number of visits, and c is the parameter balancing exploitation and exploration. The left term in the parentheses represents the

child node with the highest average reward from past trials (exploitation), while the right term represents the child node that has been explored less frequently (exploration).

This method ensures that the generated payloads are syntactically correct and possess a greater ability to bypass stringent WAFs. Its drawback is the significant computational overhead, requiring extensive interaction with the target WAF to accumulate experience.

B. Search-based Methods

The search-based method is inspired by Adaptive Random Testing (ART) [13], which defines the 'distance' between any two members of the input domain to describe the likelihood of them exhibiting similar faulty behavior. ART assumes that the closer the input points are, the higher the probability that they share the same fault. Therefore, if a test case does not trigger a fault, the next test case should be selected to be as far away as possible from the existing test cases, as shown in (3):

$$t_{k+1} = arg\max_{t \in I} (\min_{t_i \in T} d(t, t_i)) \tag{3}$$

Where I is the input space, T is the tested case set $T = \{t_1, t_2, ..., t_k\}$, d is the distance function.

In the problem of SQLi payload search, researchers focus on defining the distance between two payloads and, based on the feedback from the target WAF, ranking or clustering payloads with higher potential for bypassing.

1) ***ML-Driven****:* Appelt et al. [1], [14] used the context-free grammar they created to randomly generate a large number of test payloads, each of which was represented as a tree structure referred to as a derivation tree. They then recursively divided the derivation trees into slices, where each slice is a subtree containing at least one leaf node (terminal symbol) and one branch point (non-terminal symbol), and each slice is assigned a unique identifier. The collection of extracted slices $P = \{s_1, s_2, ..., s_N\}$ is used to represent a payload. Based on whether the payload is blocked by the target WAF, a random forest classifier is used to predict which slices and slice combinations can evade the target WAF. Based on the trained random forest classifier, they ranked all payloads according to their probability of evading the target WAF, prioritizing those that were blocked but had high potential for mutation. They employed the $(\mu + \lambda)$ Evolutionary Algorithm (EA) to iterate continuously, selecting μ payloads from parent payloads and λ mutant payloads as the next generation of payloads.

2) ***ART4SQLi****:* Zhang et al. [4] based their approach on the observation that effective payloads tend to cluster together. As a result, they employed traditional ART methods to select samples that are distant from ineffective payloads. To achieve this, they utilized context-free grammars to decompose payloads and characterized each payload by the frequency of each terminal symbol appearing within it. Specifically, they transformed each payload into a normalized k-dimensional vector $v_p = \left[\omega_p^1, \omega_p^2, ..., \omega_p^k\right]$, where k is the number of terminal symbols, and each component is defined as shown in (4):

$$\omega_p^i = \frac{\log\left(F_p^i + 1.0\right) \times \log\left(\frac{k}{N^i}\right)}{\sqrt{\sum_{i=1}^{k} \left[\log\left(F_p^i + 1.0\right) \times \log\left(\frac{k}{N^i}\right)\right]^2}} \tag{4}$$

Where F_p^i represents the frequency of the terminal symbol t_i in the payload, and N^i denotes the number of payloads containing t_i. The term $\log\left(F_p^i + 1.0\right) \times \log\left(\frac{k}{N^i}\right)$ takes into account both the local frequency of t_i and its global rarity, serving as a measure of the importance of t_i.

Subsequently, they measured the distance $\mathcal{D}$ between two payloads by calculating the cosine similarity between their vectorized representations. For each vector in the candidate set CS, the minimum cosine similarity between it and all vectors in the evaluation set ES is computed. Then, among these minimum similarities, the payload p associated with the maximum value is selected as the next payload to test:

$$p = \max_{u \in CS} \left\{ \min_{v \in ES} \{\mathcal{D}(u, v)\} \right\} \tag{5}$$

3) ***RAT****:* Amouei et al. [5] proposed clustering similar payloads first and then focusing the search effort on the effective clusters. They tokenize payloads using special characters and SQL keywords, then apply n-gram techniques to extract token combinations, referred to as fragments. These fragments are vectorized using Word2Vec. Utilizing a cosine distance measure, they cluster the fragments and convert a payload p into a binary vector that represents the presence or absence of fragment clusters. For example, $p = [1, 0, 1]$ indicates that the payload contains the 1st and 3rd clusters but not the 2nd. They argued that clustering high-dimensional binary vectors based on similarity measures leads to poor performance. Therefore, they employed a Deep Embedding Network (DEN) to learn low-dimensional representations of these vectors, and finally applied the K-Means algorithm to cluster the low-dimensional features, which results in the payload clustering.

In order to select the most promising payload in each payload cluster, RAT first removes fragments that do not appear in the cluster. It then calculates the entropy of each fragment and removes highly repetitive and extremely rare fragments (non-informative fragments). The entropy is calculated using the formula shown in (6):

$$E(f_i) = -\left(\frac{k_i}{N}\log_2\frac{k_i}{N} + \frac{N - k_i}{N}\log_2\frac{N - k_i}{N}\right) \tag{6}$$

Where E represents the entropy, i is the fragment index, N is the number of payloads in the cluster, and k_i is the number of payloads containing fragment f_i.

They argue that among the remaining fragments, rare fragments are more valuable. Therefore, they use inverse document frequency $\omega_f^i = \ln\frac{N}{k_i}$ [15] to calculate the weight of each unique fragment ω_f^i. For each payload, they create a feature vector of length n, where n is the number of fragments, and each element represents the corresponding fragment's weight. The score is then computed and ranked using the following formula:

$$score(p) = \sum_{i=1}^{n} b_i \times v_i \tag{7}$$

Where i is the fragment index, b_i is the frequency of fragment f_i in blocked samples, and v_i is the feature vector of payload p.

By clustering payloads and scoring them within each cluster, RAT effectively improves the accuracy of the search. Additionally, they adopted a ε-greedy policy to select the most valuable clusters.

C. Generative-based Methods

Generative-based methods draw inspiration from techniques in natural language processing (NLP). In recent years, Transformer models have demonstrated exceptional performance in natural language generation, excelling at producing text that is both grammatically and semantically coherent within context. Their application in generating SQLi payloads is expected to become a common approach in the future. **GPTFuzzer**, as proposed by Liang et al. [2], first tokenized each payload in the SQLi dataset, converting it into an integer sequence $\tau = \{x_0, x_1, ..., x_T\}$, where x_0 and x_T represent the $\langle$start$\rangle$ and $\langle$end$\rangle$ tokens, respectively. They trained a multi-layer Transformer decoder to learn the sequence probability distribution ρ, while introducing a reinforcement learning strategy to optimize the generative model, defining each action as the selection of a valid token and receiving a reward R_t:

$$R_t = \begin{cases} -\beta\, KL_t(\pi_{\theta_k}, \rho), & \text{for } t \in \{0, \ldots, T-1\} \\ R_T^{\text{WAF}}, & \text{for } t = T \end{cases} \tag{8}$$

Where adjusting β allows a trade-off between bypass rate and diversity, and KL_t quantifies the difference between the probability distribution given by the policy π and that of the pre-trained model ρ, preventing excessive deviation of the policy π from the language model ρ:

$$KL_t(\pi_{\theta k}, \rho) = \log \frac{\pi_{\theta_k}(x_{t+1} \,|x_{0:t}\,)}{\rho(x_{t+1} \,|x_{0:t}\,)} \tag{9}$$

Where $\rho(x_{t+1}\,|x_{0:t}\,)$ represents the probability of the next token x_{t+1} given the current state $x_{0:t}$ as generated by the pre-trained model ρ. Additionally, $\pi_{\theta_k}(x_{t+1}\,|x_{0:t}\,)$ represents the probability of selecting x_{t+1} at the current state.

D. Detection Methods

Although adversarial SQLi has not yet garnered widespread attention, traditional SQL injection, recognized as one of the top ten security risks by OWASP[1], has led to the development of numerous methods for detecting attack payloads. These methods can be broadly categorized into three types: rule-based matching, handcrafted feature extraction, and content-based approaches.

1) Rule Matching-Based: Rule-based detection methods assume that the characteristics of malicious requests can be described using predefined rules. Whenever a new request is received, it is matched against the rule library to determine whether it is malicious. For example, Denning [16] first constructed a corresponding rule library in 1987 by statistically analyzing the anomalous behavior of existing malicious traffic, identifying requests similar to the anomalies in the rule library as malicious. A widely used open-source WAF, ModSecurity[2], operates on this principle by creating a core rule set to provide protection against common vulnerabilities. Additionally, there are whitelist-based WAFs like Naxsi[3], which identify any anomalous behavior by analyzing the expected behavior of normal traffic and flag it as intrusive [17].

2) Handcrafted Feature Extraction-Based: In detection methods based on handcrafted feature extraction, researchers typically select various custom features based on empirical knowledge to construct feature vectors, which are then classified using machine learning algorithms. For example, Tang et al. [18] extracted statistical features such as payload length, keyword count, and special character count from URLs, and then classified them using a Multi-Layer Perceptron (MLP), demonstrating a simple yet efficient approach. Ramezany et al. [19] computed TF-IDF using n-grams of different scales and selected the top-k features to construct feature vectors. Zhou et al. [20] replaced all digits in the payload with 'A' and all letters with 'D', and obtained feature vectors by performing frequency statistics on patterns occurring in the simplified strings.

3) Content Detection-Based: Advancements in deep learning and NLP have driven growing attention to content-based detection methods. Researchers have explored learning syntactic and semantic patterns in payloads via tokenization, word embeddings, and neural network-based feature extraction: Yu et al. [21] and Kim et al. [22] used multi-kernel TextCNN for multi-scale local features, while Wang et al. [23] and Tadhani et al. [24] employed LSTM to capture sequential temporal dependencies and subtle contextual patterns. Some other studies treat payloads as text data, leveraging NLP techniques for semantic feature extraction and attack detection. Bokolo et al. [25] used Transformer tokenizers to embed URL/Referer fields into word vectors, with pre-trained BERT for classification. Salim et al. [26] evaluated Robustly Optimized BERT and DistilBERT, showing these natural language pre-trained models achieve 80% accuracy on malicious request detection with only payload inputs and no expert knowledge. Additionally, some works adopt an anomaly detection perspective: Tang et al. [27] trained an AutoEncoder to model benign payload patterns, identifying malicious payloads as anomalies via larger reconstruction errors.

III. The Proposed Method

A. Motivation

Adversarial SQLi has evolved into a sophisticated threat with the emergence of automated tools mentioned above,

[1]https://owasp.org/www-project-top-ten/

[2]https://github.com/owasp-modsecurity/ModSecurity

[3]https://github.com/wargio/naxsi

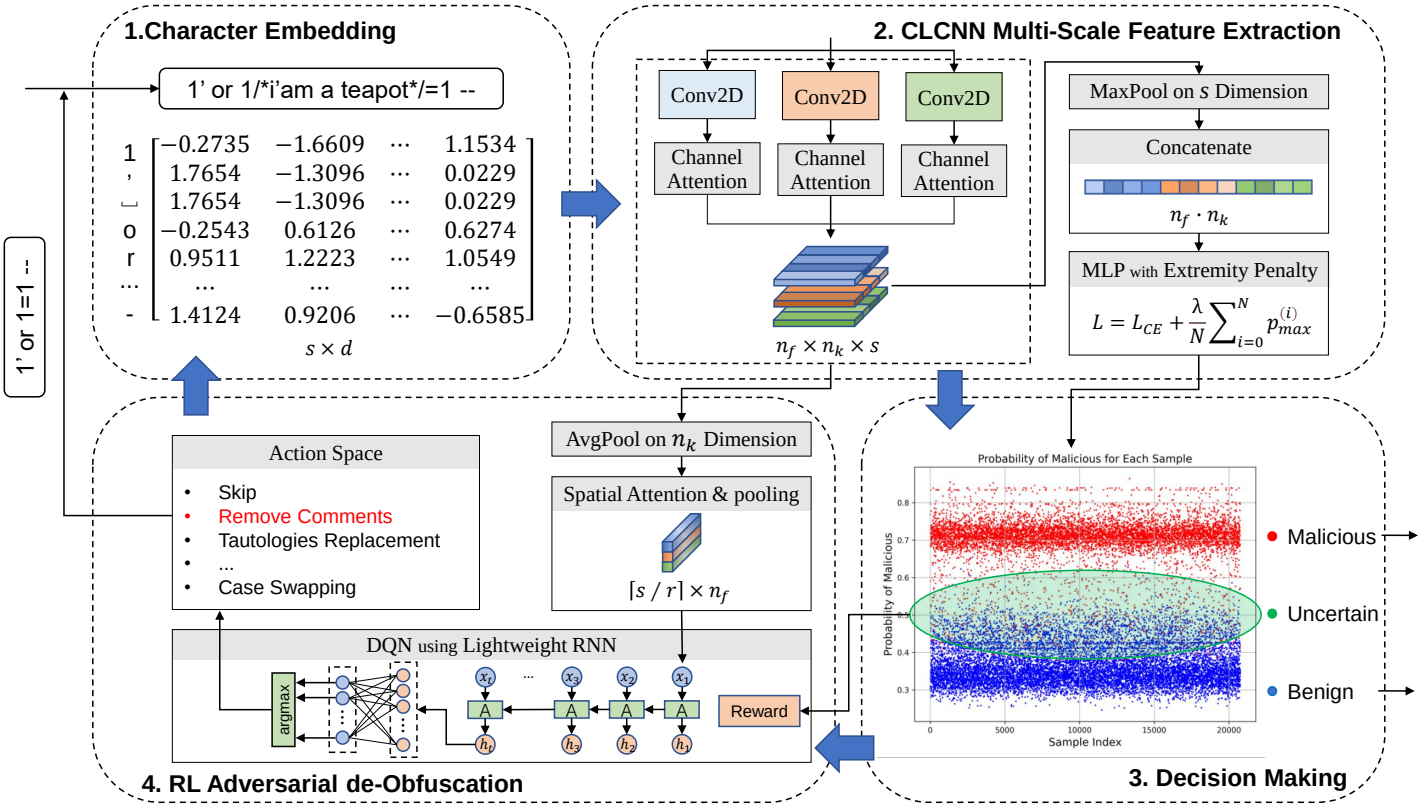


Fig. 2. Framework of the Hybrid Model. It performs character-level embedding on the character sequences of the payloads, followed by multi-scale feature extraction and channel attention weighting using the CLCNN. The output is then processed by a multi-layer perceptron (MLP) to produce a prediction, which determines whether to accept the payload. If the prediction falls within the uncertain range, further pooling and spatial attention weighting are applied to the feature maps output by the CLCNN. Subsequently, an RNN extracts sequential features to select appropriate de-obfuscation operations from the action space. After executing the de-obfuscation operation on the payload, this process is repeated until a final decision is made.

which dynamically generate obfuscated payloads to bypass WAFs. These tools leverage semantic-preserving mutations (summarized in Table I) to produce a large number of adversarial variants, posing unprecedented challenges to existing detection methods. This section systematically analyzes the core challenges of adversarial SQLi and clarifies the motivation for proposing the CLCNN+RL hybrid model. Adversarial SQLi tools employ three primary strategies to evade detection, each introducing unique challenges for WAFs:

1) **Syntactic Structure Destruction:** Mutation operations such as recursive encoding, nested comments, and keyword substitution break the inherent syntactic structure of malicious patterns. Traditional NLP-based detection methods rely on tokenization or syntactic parsing, which fail when obfuscations disrupt predefined token boundaries or grammatical rules.

2) **Feature Dilution and Obfuscation:** Adversarial payloads insert redundant characters, apply multi-layer encoding, or perform logical invariant transformations, diluting the original malicious features into irrelevant noise. Rule-based WAFs are unable to match obfuscated patterns not covered by predefined rules, while statistical feature-based models struggle to distinguish effective malicious features from obfuscation-induced noise.

3) **Dynamic Adaptability:** Adversarial SQLi attacks dynamically adjust their strategies based on real-time WAF feedback. Attackers continuously iterate and refine payloads to evade static detection logic [28]. In contrast, most existing detection models are static (or have long update cycles), lacking the ability to promptly adapt to evolving attack strategies. This inherent asymmetry between dynamic attacks and static defenses creates persistent vulnerabilities that adversaries can exploit, posing a fundamental challenge to traditional WAFs.

To address these issues, we propose a hybrid model combining Character-Level CNN and Reinforcement Learning, with design motivations tailored to counter the core challenges of adversarial SQLi. Unlike word-level models, CLCNN processes payloads as continuous character sequences without relying on tokenization or syntactic parsing. From a pattern recognition perspective, it learns low-level malicious patterns such as characteristic character combinations and abnormal character distributions that remain preserved across obfuscations. From an anomaly detection perspective, it captures obfuscation-induced abnormal character sequences including recursive encoding and nested comments as auxiliary features, which effectively mitigates feature dilution. Equipped with multi-scale convolutional kernels and attention mechanisms, CLCNN further enhances its ability to capture fragmented malicious patterns, laying a robust foundation for static feature extraction.

Complementing CLCNN's static capabilities, the RL mod-

ule is designed to inject critical dynamic adaptability into the framework: for highly obfuscated payloads that may still obscure malicious patterns, the RL module performs targeted, sequential de-obfuscation operations on ambiguous payloads, embodying dynamic processing that aligns with the iterative nature of adversarial attacks. More importantly, as a plug-and-play component, the RL module can be frequently updated to adapt to emerging attack tactics without modifying the core CLCNN architecture—this ensures that the static detection performance of CLCNN (validated for standard and known attacks) remains stable [29], while the RL module continuously evolves to counter new obfuscation strategies. This plug-and-play design also endows the framework with strong scalability across other Web attack detection tasks (e.g., XSS, command injection). For migration, one only needs to select a high-performance machine learning model for the target domain, take the hidden layer output of this target model as the input of the RL module (eliminating the need for independent feature generalization efforts), freeze the target model's parameters, and retrain the RL module following our proposed framework. The action space of the RL module can be updated by inverting the semantic-preserving mutation operators of adversarial attacks in the target domain, directly forming domain-specific de-obfuscation actions.

By integrating CLCNN's robust static pattern recognition and anomaly detection with the RL module's dynamic de-obfuscation and adaptive learning capabilities, the hybrid model achieves comprehensive defense against both known and unseen adversarial SQLi variants, effectively addressing the syntactic destruction, feature obfuscation, and dynamic adaptability challenges posed by modern adversarial attacks. The subsequent sections detail the implementation of each module and their collaborative mechanism.

B. Framework

As shown in Fig. 2, our hybrid model primarily consists of a multi-scale feature extraction module based on CLCNN, an adversarial de-obfuscation module based on RL, and a decision module. After extracting effective payloads, the hybrid model first performs preprocessing and character-level embedding, followed by the feature extraction module, which utilizes multiple convolutional kernels of different scales to extract local features from the character sequence. The decision module interprets the feature map from various perspectives to make predictions and determine whether de-obfuscation is needed. On one hand, through max pooling, the decision module obtains the maximum activation value for each kernel as input to the MLP, yielding confidence outputs. On the other hand, if the confidence is insufficient, the decision module applies average pooling to the feature map to obtain the average activation value of each filter at each character position, which serves as the state for the reinforcement learning module. The confidence is also incorporated as part of the reinforcement learning module's environment to compute the reward for training, enabling the reinforcement learning module to appropriately select de-obfuscation actions from the action space based on the state, thereby removing obfuscation from the original payload. This cycle continues until the decision module achieves sufficient confidence or times out.

C. Character-Level CNN

Unlike natural language, researchers processing web request payloads must tokenize based on special characters or regular expressions rather than simply using spaces as delimiters. Traditional tokenization methods struggle with irregular inputs that have been deliberately obfuscated, and SQLi mutation operations further obscure the original payload's semantics. To address this issue, we adopted CLCNN because it does not rely on specific tokenization rules or syntactic structures, allowing it to handle inputs from various languages and encodings. Our CLCNN is similar to that proposed by Saxe [30] and Ito [31], as both approaches utilize convolutional kernels of multiple scales to extract local features from character sequences. But this study adopts a pattern matching perspective, with the expectation that each randomly initialized kernel will learn a specific pattern. The specific processes are as follows:

1) Preprocessing: Before training, we only decode a portion of the data for which the encoding method is clearly knonwn, avoiding excessive preprocessing, as data blocks resulting from malicious obfuscation, such as recursive encoding, are effective features in themselves. Additionally, we extract the valid portions from various publicly available datasets, such as the GET and POST parameters, headers, or request bodies in HTTP requests, which we refer to as payloads. All subsequent SQL injection detection efforts will revolve around these payloads.

2) Character Embedding: To convert request payloads into a computable format, we used character embedding. First, we performed character encoding on payloads, mapping each character to its unique ASCII numerical value. The ASCII table includes 95 printable and 33 control characters; since control characters are absent from request payloads, valid character indices range from 1 to 95, with 0 denoting unexpected characters (e.g., from transmission or decoding errors). Next, we mapped all payloads to integer sequences per the above rules, padding with 0 or truncating to a uniform length s. We then used the PyTorch `nn.Embedding` module to convert these sequences into continuous vector representations, yielding tensors of size $s \times d$ (where d is the character embedding dimension). The `nn.Embedding` module initializes a fixed-size embedding matrix based on vocabulary size and embedding dimension, with each row corresponding to a unique integer index retrievable via index lookup. During training, our CLCNN extracts multi-scale features from the embedding vectors and updates the embedding matrix weights via backpropagation. Unlike CBOW and Skip-gram, this approach does not depend on a fixed context window and instead captures fine-grained character-level local features.

3) Feature Extraction: In our approach, we apply n_f filters (width d, lengths $k \in \{1, 3, 5, 7, 9\}$) to the $s \times d$ embedding vectors. Each scale filter contains n_k randomly initialized kernels, intended to identify salient patterns across payload categories. Since the kernel width matches the character embedding dimension, the kernel slides in only one direction

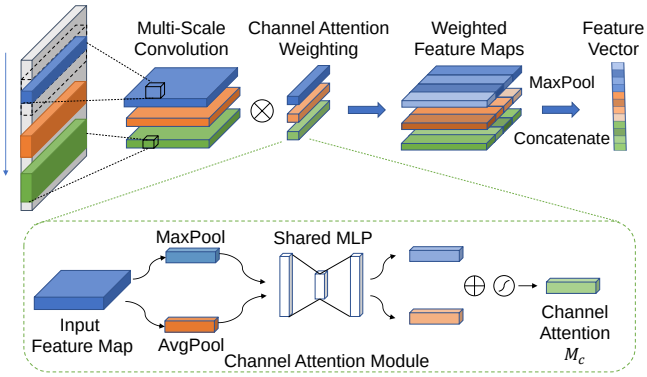


Fig. 3. Illustration of Multi-Scale Feature Extraction and Channel Attention Weighting.

during feature extraction, outputting a larger activation value at the corresponding position in the feature vector when matching the target pattern. To ensure consistent lengths for feature vectors output by kernels of different sizes, we apply padding to both ends of the input data for each convolutional kernel, with the padding value $pad = \frac{k-1}{2}$ (where k is odd). Following this padding and convolution with a stride of 1, each kernel yields a feature vector of length s, where the i-th value represents the kernel's activation centered on the i-th character of the original payload. This value indicates the similarity between the substring in the range $[i-k, i+k]$ and the pattern learned by the kernel. We then stack outputs from different filters to obtain a feature map of dimensions $n_f \times n_k \times s$, where n_f is the number of filters, n_k is the number of kernels per filter, and s is the input sequence length.

Considering that each kernel contributes differently to the decision-making process, we introduce the channel attention mechanism [32] to weight the outputs based on the importance of the kernels. As shown in Fig. 3, for the feature map $M \in \mathbb{R}^{n_k \times s}$ produced by each scale of the filters, we utilize max pooling and average pooling to obtain two distinct context descriptors $F^C_{max} \in \mathbb{R}^{n_k \times 1}$ and $F^C_{avg} \in \mathbb{R}^{n_k \times 1}$. Subsequently, we employ a shared MLP with one hidden layer to generate the channel weights:

$$M_c = \sigma(MLP(F^C_{max}) + MLP(F^C_{avg})) \tag{10}$$

where σ denotes the sigmoid function, and $M_c \in \mathbb{R}^{n_k \times 1}$ is the weight vector. The weighted feature map M' is obtained by performing channel-wise multiplication:

$$M' = M \odot M_c \tag{11}$$

Channel attention weighting for each filter scale is independent, enabling the model to adaptively adjust the importance of scale-specific features and better fuse multi-scale information in subsequent processing. For the task of predicting payload maliciousness probability, we perform max pooling along the final dimension to obtain a tensor of size $n_f \times n_k$. Each value in this tensor represents the maximum activation of a kernel sliding over the character sequence, indicating the presence of the kernel's learned pattern in the sequence.

In contrast, for providing state inputs to RL module, we must consider not only the presence of patterns but also their identities and contextual relationships. Thus, we perform average pooling along the kernel dimension to yield a tensor of size $n_f \times s$. Each element in a row denotes the average activation of a filter at that position, and the i-th column (a combination of average activations across all filter scales) represents the pattern at the i-th character position. We then introduce the spatial attention mechanism [32], allowing the model to emphasize or suppress specific patterns based on contextual relationships and thereby enhance feature extraction effectiveness. As shown in Fig. 4, for the feature map $M \in \mathbb{R}^{n_f \times s}$, we use both max pooling and average pooling to obtain two distinct context descriptors $F^S_{max} \in \mathbb{R}^{1 \times s}$ and $F^S_{avg} \in \mathbb{R}^{1 \times s}$. We concatenate them along the first dimension to form $[F^S_{max}, F^S_{avg}] \in \mathbb{R}^{2 \times s}$, and subsequently compute the spatial attention weights using a convolutional layer:

$$M_s = \sigma(Conv([F^S_{max}, F^S_{avg}])) \tag{12}$$

Where σ is the Sigmoid function, and $M_s \in \mathbb{R}^{1 \times s}$ is the weight vector. The weighted feature map can be obtained through element-wise multiplication, consistent with Equation (11).

4) Decision Making: Although CLCNN exhibits some resistance to obfuscated SQLi attacks due to their fine-grained nature and lack of tokenization, it remain vulnerable to highly obfuscated and meticulously crafted attacks. These payloads do not completely erase malicious patterns, but they lead to existing models being unable to provide sufficiently high confidence levels. However, easily lowering the threshold for binary classification may significantly increase the false positive rate. Therefore, we consider delineating an uncertain region and allowing a reinforcement learning module to handle these payloads for which the CLCNN lacks sufficient confidence. To distinguish between benign, malicious, and uncertain regions, rather than allowing the predicted values to approach the extremes of 0 or 1, we introduced an additional penalty term into the loss function, as shown in (13):

$$L = L_{CE} + \frac{\lambda}{N} \sum_{i=0}^{N} p^{(i)}_{\max} \tag{13}$$

Where L_{CE} represents the Cross-Entropy loss function, λ is a hyperparameter, and $p_{\max}$ is the maximum value in the probability vector output by the sigmoid function.

D. Reinforcement Learning

Relying solely on traditional neural networks may not successfully detect highly obfuscated SQLi payloads, as they obscure the original features. However, if a SQLi payload retains its malicious functionality, it must still carry certain key patterns, although they are concealed. Therefore, we defined a set of de-obfuscation operators to enhance the features of these key patterns. Given that different sequences and combinations of de-obfuscation operations can lead to varying changes in the payload, it is challenging to manually devise a de-obfuscation strategy that fits all scenarios. Thus, we introduced reinforcement learning to address this issue.

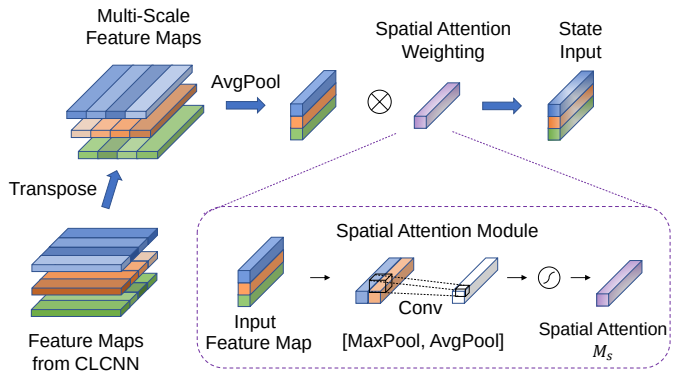


Fig. 4. Illustration of Average Pooling and Spatial Attention Weighting.

Reinforcement learning involves training an agent to make decisions through trial and error within an environment, with the objective of maximizing long-term rewards. In our context, consider a robot responsible for string manipulation, capable of performing a set of operations to modify strings. Rather than specifying which operations to apply when receiving a string, we provide rewards based on the classification confidence of the modified string. This made the agent to favor operations that enhance SQLi detection accuracy for specific strings.

1) Definition: We formalize the SQLi de-obfuscation problem using the Markov Decision Process (MDP) [33], a classical framework for solving sequential decision-making problems. In the SQLi de-obfuscation problem, the payload is derived from modifications to the previous payload, and the state is explicitly provided by the convolutional layers of CLCNN. This conforms to the MDP requirements that the environment being Markovian and fully observable. The MDP can be represented by the following four-tuple (S, A, T, R) [12], [34]:

- S is a set of states offered by the environment. In our problem, the feature vector for each payload represents a state.
- A is a set of actions that modify the payload, which may lead to a transition between states. The policy $\pi : S \rightarrow A$ denotes the probability distribution of action selection given a specific state.
- $T(s, a) : S \times A \rightarrow S'$ is the state transition function, which represents the probability of transitioning to state s' after taking action a in state s. This corresponds to the potential changes in the payload after it has been edited. A trajectory τ is expressed as $(s_0, a_0, s_1, a_1, ..., a_{t-1}, s_t)$ to represent the state transitions occurring during the detection process of a payload.
- $R(s, a, s') : S \times A \times S' \rightarrow R$ is the reward function, which indicates the reward received by the agent after taking action a in state s and transitioning to state s'. This corresponds to whether the modified SQLi payload is more conducive to detection by the model.

Given this framework, our objective is to optimize the policy π to maximize the cumulative reward in the MDP. Since our action space is discrete, this task is particularly well-suited for value-based methods, which optimize the value function to indirectly guide the policy.

The state-action value function $Q(s, a)$, a well-known concept in reinforcement learning, is defined as follows:

$$Q(s, a) = E\left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \,|s_0 = s, a_0 = a\right] \tag{14}$$

Here, E denotes the expectation, γ $(where\ 0 < \gamma < 1)$ is the discount factor used to reduce the value of future rewards (considering that rewards further in the future are less valuable). r_{t+1} represents the reward obtained by taking action a in state s_t at time t and transitioning to state s_{t+1}. Consequently,The Q function calculates the expected value of future rewards given the state s and action a, capturing the long-term benefit. Thus, the policy can be defined as selecting the action with the maximum Q-value:

$$\pi(s) = \arg\max_a Q(s, a) \tag{15}$$

Therefore, our objective is to update the Q-function so that it gradually approaches the expected future returns. This means that, for each state-action pair, the value of the Q-function accurately reflects the cumulative rewards obtained by taking that action in the given state.

2) Method: To train the Q-function, one intuitive approach is to allow the agent to interact with the environment multiple times, obtaining several trajectories, which directly provide the target values for the Q-function (Monte-Carlo, MC). But to improve the training efficiency of the model, we use the Temporal-Difference (TD) method [35]. This method incorporates the idea of dynamic programming, updating the Q-function at each step without requiring a complete trajectory. The update formula is as follows:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)) \tag{16}$$

In this formula, α is the learning rate, γ is the discount factor, and r represents the reward obtained after taking the current action. $\max_{a'} Q(s', a')$ denotes the maximum expected reward after transitioning to state s'. The goal of this formula is to make $\max_{a'} Q(s', a') - Q(s, a)$ approach the immediate reward r provided by the environment, in order to estimate the Q-value.

The algorithm's core process is straightforward: the agent takes the original SQLi payload, uses the CLCNN's output feature vector as the state, selects and executes actions based on computed Q-values, and receives rewards from the environment based on the modified payload's detection confidence. The Q-function is then updated using these rewards (per Equation (16)) to represent expected returns. However, Q-value computation poses a challenge due to the near-infinite state space, making it impractical to assign Q-values to all state-action pairs. We thus use Deep Q-Networks (DQN) to approximate the Q-function—employing an RNN for sequential feature processing and a fully connected layer to output raw action scores. To address Q-value overestimation [36], we adopt Double DQN, where one Q-function selects actions and the other evaluates their values. Additionally, we incorporate Prioritized Experience Replay and Target Network training (as

in [37]), with DQN parameters updated following Algorithm 1:

Algorithm 1 The Training Algorithm

Initialize the main Q-network $Q(s,a)$ and the target Q-network $Q'(s,a)$, setting $(\theta' = \theta)$.
Initialize the experience replay buffer D.
for $episode = 1, 2, ..., N$ do
  env.reset(); $count$ = 0.
  Get the input payload p and its $label$.
  $s, predict = CLCNN(p)$.
  if $predict[label] > \beta_1$ **then**
    continue;
  end if
  for $t = 1, 2, ..., T$ **do**
    $a = \begin{cases} random & with\ probability\ \varepsilon \\ \arg\max_a Q(s,a) & with\ probability\ 1-\varepsilon \end{cases}$
    $p^* = mutate(p, a)$.
    $s^*, predict^* = CLCNN(p^*)$.
    $\delta = (predict^* - predict)[label], \quad -1 \le \delta \le 1$.
    $r = \beta_2(\exp(\delta) - 1)$.
    if $predict^*[label] > \beta_1$ **then**
      $done = True$.
    else
      $done = False$.
    end if
    $td\_error = \begin{cases} r - Q(s,a) & if\ done \\ r + \gamma \max_{a'} Q'(s',a') - Q(s,a) & else \end{cases}$
    $priority = (abs(td\_error) + 10^{-5}) * \beta_3$.
    $D.add(s, a, r, s', done, priority)$.
    if $len(D) > \beta_4$ **then**
      Draw a batch of samples according to:
      $P_{sample} = priority / sum(priority)$.
      For each sample, compute the target Q-value:
      $y = r + (1 - done)\gamma Q'(s', \arg\max_{a'} Q(s', a'))$.
      Define the loss function $L(\theta)$ as:
      $L(\theta) = E_{(s,a,r,s',done)\sim D}\left[(y - Q(s,a))^2\right]$.
      $\theta \leftarrow \theta - \alpha \nabla L(\theta)$.
      $count \leftarrow count + 1$.
      if $count > \beta_5$ **then**
        $\theta' \leftarrow \theta$.
      end if
    end if
    $predict \leftarrow predict^*$.
    $p \leftarrow p^*$.
    if done == True **then**
      break
    end if
  end for

3) Action Space: In the initial state or after each transition to a new state,the agent will select and execute the action with the highest expected reward from the action space, based on the output of the Q-network. The action space is as follows:

1) Skip: Do not perform any operations and directly use the current prediction value as the output of the hybrid model.
2) Remove Comments: Match and remove block comments (/* */) and their contents. Match and remove the content commented out by the last line comments(-- or #)
3) Replace Comments: Match and replace block comments with /*reg*/. For MySQL-specific conditional comments (/*! ... */), retain the content following the exclamation mark.
4) Placeholder Replacement: Search and replace placeholders such as '\n' or '\t' with a space.
5) DML Substitution: Search and replace operators, for example, replacing '||' with 'OR'.
6) Simplify Expressions: For longer executable Boolean expressions, provide the simplified result directly. For example, replace 'A AND 1=1 OR 2=1' with 'A', where 'A' represents expressions that cannot be directly evaluated as true or false.
7) SubQuery Convert: Convert the harmless subquey to true. For example, convert '(SELECT 1)' to '1 = 1'
8) Integer Decoding: Search for integers encoded as Base-64 or Hexadecimal and replace them with decimal numbers (or integer 1).
9) URL Decoding: Match and decode URL-encoded strings. For example, decoding '%20' to '!'.
10) Case Swapping: Tokenize using special characters and replace certain keywords with their uppercase forms, such as replacing 'SeLEct' with 'SELECT'.

The above actions can be uniformly formulated as regex-based targeted search and replacement operations. To clarify the action space design logic, a core principle is highlighted: each de-obfuscation action is explicitly mapped to the semantic-preserving mutation operations summarized in Table I. This one-to-one correspondence ensures de-obfuscation directly counteracts the obfuscation strategies of adversarial SQLi tools, maximizing the exposure of concealed malicious patterns. Notably, the RL module is only invoked when the CLCNN output falls within the uncertain interval [0.4, 0.6]. The "Skip" action within the RL action space serves two key purposes: it avoids misleading artifacts caused by inappropriate de-obfuscation operations, and provides a direct decision-making channel that circumvents redundant iterative attempts. When "Skip" is selected, the model directly outputs the classification result using 0.5 as the decision boundary based on the current CLCNN output, effectively balancing detection efficiency and accuracy.

IV. Evaluation

To validate the effectiveness of our hybrid model, we selected three open-source WAFs with default configurations, along with three well-known and reproducible machine learning-based SQLi detection methods as baselines. We conducted a series of comparative experiments to evaluate our model's comprehensive performance in defending against adversarial SQLi attacks. Specifically, this section aims to address three key questions:

- **RQ1:** Do adversarial SQLi attacks pose a significant threat? What challenges do the three categories of attacks present?

- **RQ2:** Is the CLCNN model not good enough? Why do we need to introduce the RL module and adversarial training?
- **RQ3:** If models undergo adversarial training against a specific adversarial SQLi tool, can they effectively resist attacks from that tool? Can multi-round adversarial training fully mitigate the tool's attacks?
- **RQ4:** What is the task allocation between the CLCNN module and the RL module? What are their respective contributions?
- **RQ5:** Can the proposed model be deployed in real-world scenarios? Are its computational overhead and resource consumption acceptable for practical deployment?

A. Aggregate Dataset

In the field of web malicious request detection, the CSIC dataset is widely used but has inherent limitations: it consists of programmatically generated web requests and is over a decade old, failing to reflect current real-world attack patterns. Moreover, many existing methods have achieved over 99% accuracy on the CSIC dataset, making it ineffective for distinguishing performance differences between various models. As noted in the literature [11], cloud service operators and WAF manufacturers typically do not disclose their business datasets (which contain real-world attack samples) or detection models, as this would compromise their competitive advantages. Therefore, to ensure the reproducibility of our work while approximating the diversity of real attack scenarios, we integrated multiple publicly available SQLi datasets and benign web request data from Kaggle [4] and GitHub [5] , including the CSIC dataset [6]. We acknowledge that public datasets may have gaps in feature distribution compared to real-world web scenarios, as they often lack the latest adversarial attack variants and practical deployment context. However, such datasets remain valuable for academic research—for example, AdvSQLi [3] successfully generated mutated payloads based on public datasets to bypass commercial WAFs, demonstrating their practical relevance.

To mitigate potential distribution imbalance and biases, we randomly sampled 120,000 benign requests and 80,000 malicious requests from the integrated dataset, dividing them into training, validation, and test sets with a 6:2:2 ratio. The inclusion of more benign requests ensures the model learns genuine malicious characteristics rather than memorizing specific patterns or biases. For instance, initial experiments revealed the CLCNN model tended to classify number-containing requests as malicious (a common trait in SQLi payloads with integers or encoded integers), but balancing benign and malicious samples helps avoid over-reliance on such superficial features, reducing false positive rates. In subsequent experiments, all models except open-source WAFs were trained on the combined training and validation sets and evaluated on the test set. Mutation-based tools mutate all test set payloads, while search-based tools limit searches to the test set. During adversarial training, payloads that bypassed target models were oversampled and added to the training set in each round, with independent adversarial training for each model.

TABLE II
PERFORMANCE OF RAW MODELS ON TEST SET.

Models	Accuracy(%)	Precision(%)	Recall(%)	F1
ModSecurity	81.63	73.77	82.01	0.7767
Naxsi	64.04	52.10	94.95	0.6728
WAF-Brain	67.62	57.06	68.02	0.6206
ANN	93.66	95.05	88.34	0.9157
LSTM	97.29	98.37	94.62	0.9646
TextCNN	97.30	**98.82**	94.21	0.9646
CLCNN	**97.36**	96.79	96.43	**0.9661**
CLCNN+RL	97.21	96.37	**96.47**	0.9642

B. Raw Model Performance

We selected three open-source WAFs representing distinct categories: ModSecurity (blacklist-based), Naxsi (whitelist-based), and WAF-Brain[7] (machine learning-based). Additionally, we adopted three baseline models: the statistical feature-based ANN [18], and the payload content-based LSTM [23] and TextCNN [21]. Open-source WAFs were deployed in Docker containers as reverse proxies protecting a simple in-container web application, with attack tools determining request blocking via HTTP response codes. Locally trained models were directly integrated into the web application, returning confidence scores for malicious classification. We define gray-box testing as scenarios where attack tools access these confidence scores; to simulate real-world conditions, we primarily used black-box testing, converting scores to binary (0/1) with a 0.5 threshold. Table II presents each model's test set performance. Locally trained models share similar parameters, and content-based detection models exhibit comparable overall performance.

The F1 score was adopted as the primary metric to evaluate the overall performance of each model on the standard test set, as it comprehensively balances Precision and Recall to reflect holistic detection capability. For adversarial attack experiments, Recall was exclusively utilized as the evaluation metric—this is attributed to the fact that all inputs in these experiments are malicious payloads (mutation-based attacks preserve the inherent malicious functionality of payloads, while search-based attacks select candidates from pre-established malicious payload libraries). Consequently, Recall is sufficient to characterize the model's resistance to adversarial tools, rendering additional metrics such as Precision unnecessary.

Specifically, the Recall calculation criteria for different adversarial attack types are defined as follows: (1) For mutation-based attacks, a malicious payload from the test set is considered correctly classified if it fails to bypass the target

[4]https://kaggle.com/datasets/syedsaqlainhussain/sql-injection-dataset
https://kaggle.com/datasets/sajid576/sql-injection-dataset
https://kaggle.com/datasets/gambleryu/biggest-sql-injection-dataset
https://kaggle.com/datasets/alextrinity/sqli-xss-dataset

[5]https://github.com/fishyyh/CNN-SQL
https://github.com/ajinmathew/SQL-data

[6]https://kaggle.com/datasets/ispangler/csic-2010-web-application-attacks

[7]https://github.com/BBVA/waf-brain

TABLE III
PERFORMANCE OF MODELS AND THE EFFECTIVENESS OF ADVERSARIAL SQLI TOOLS.

Test Model	Recall(%)	'Recall'(%) on Mutated Test Set			Recall(%) on Adaptive Test Set		Recall(%) on Generated New Set	
		WAF-A-MoLE	AdvSQLi(R)	AdvSQLi	ART4SQLi	RAT	ML-Driven	GPTFuzzer
ModSecurity	**82.01**	80.44	80.69	76.31	76.60	**51.57**	99.97	63.93
Naxsi	**94.95**	94.18	92.98	92.12	92.92	**66.77**	99.79	99.95
WAF-Brain	**68.02**	47.15	36.05	**9.60**	54.69	48.45	70.59	60.87
ANN	**88.34**	76.86	75.78	**59.40**	81.62	71.88	90.30	81.91
LSTM	**94.62**	78.53	84.41	**57.42**	90.02	83.25	67.95	75.40
TextCNN	**94.21**	74.81	84.26	**51.64**	89.09	82.31	71.67	59.03
CLCNN	**96.43**	**89.49**	94.32	92.50	93.80	96.16	99.81	99.99
CLCNN+RL	**96.47**	**90.89**	95.56	93.03	93.38	96.15	99.44	99.98

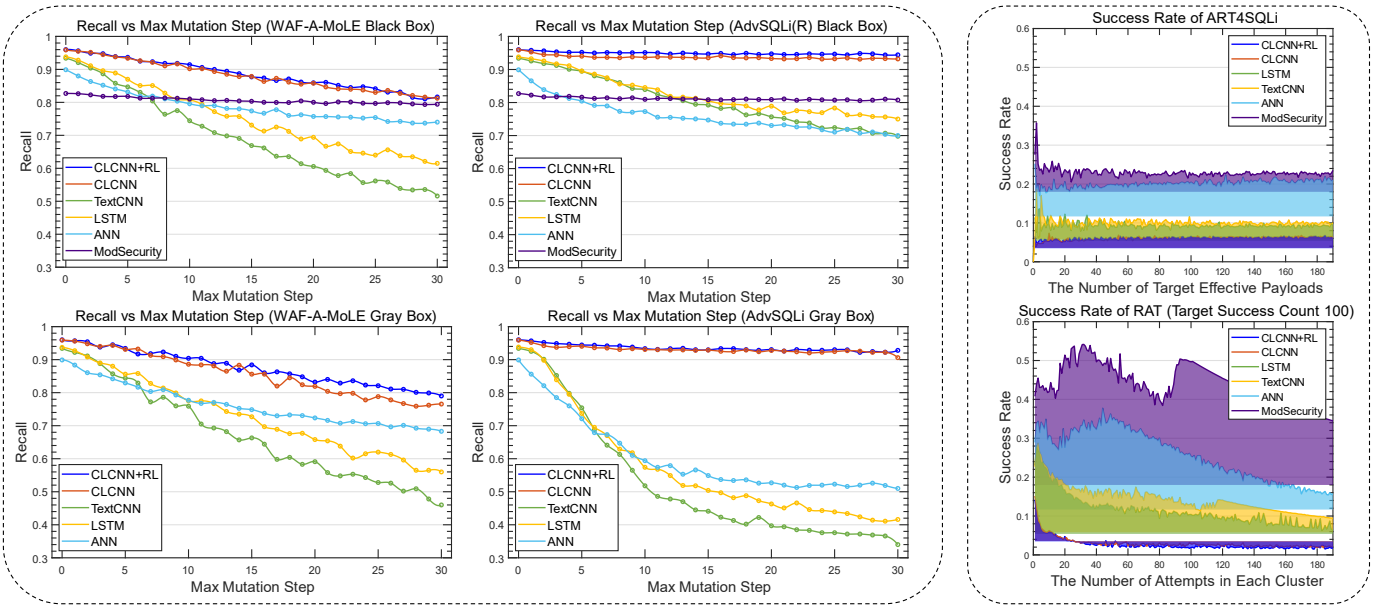


Fig. 5. Illustration of the Performance Metrics of the Raw Model Against Various Adversarial SQLi Attacks. The left panel shows the Recall of each model decreases as the maximum number of mutations increases when confronted with mutation-based tools. The right panel demonstrates the improvement in success rates of search-based tools across the models compared to random selection. Note that AdvSQLi(R) denotes random mutations (10 times total) using AdvSQLi's mutation methods without strategic selection.

WAF even after undergoing up to 10 mutation operations. (2) For search-based methods, the calculation of Recall only considers payloads prior to the first 100 successful attempts at bypassing the target WAF. (3) For generation-based attacks, model performance is assessed on the generated payload dataset. Notably, ML-Driven is categorized as a generation-based attack in this study, as it automatically generates initial attack payloads from context-free grammars before performing subsequent mutations. For GPTFuzzer, we directly utilized the publicly available dataset provided by its authors instead of conducting retraining from scratch.

The experimental results are summarized in Table III, where all values represent the Recall of corresponding models under the respective adversarial tools listed in the columns. Bold-faced values highlight the maximum Recall reduction observed for a single model across different attack scenarios. It should be noted that data in the columns for WAF-A-MoLE and AdvSQLi were collected under a gray-box testing scenario, with the *computational budget* for AdvSQLi set to 1 to ensure fair comparison; all other data were obtained under a black-box scenario. To guarantee the reliability and reproducibility of the results, all experiments were repeated at least 3 times, with search-based attack experiments replicated no fewer than 15 times. The final reported values are the averages derived from all repeated trials.

1) Raw Baseline Model Performance: Rule-based WAFs (ModSecurity and Naxsi) were minimally impacted by mutation-based attacks. While Naxsi's strict whitelist leads to a high false positive rate, this trait enhances its resilience to such attacks. However, search-based attacks effectively exploit their vulnerabilities: despite Naxsi's initial 94.95% Recall, RAT quickly locates the remaining 5.05% of effective payloads, reducing its Recall to 66.77%, and ModSecurity's Recall drops by nearly 30%. Additionally, ModSecurity's Recall falls to 63.93% against the adversarial GPTFuzzer dataset. Machine learning-based models faced greater challenges: default-configured WAF-Brain's Recall plummets to 9.6%. The locally trained ANN, LSTM, and TextCNN models experience 40% Recall declines against AdvSQLi, with a 10% drop even after 10 random mutations. Since mutation-based tools perform

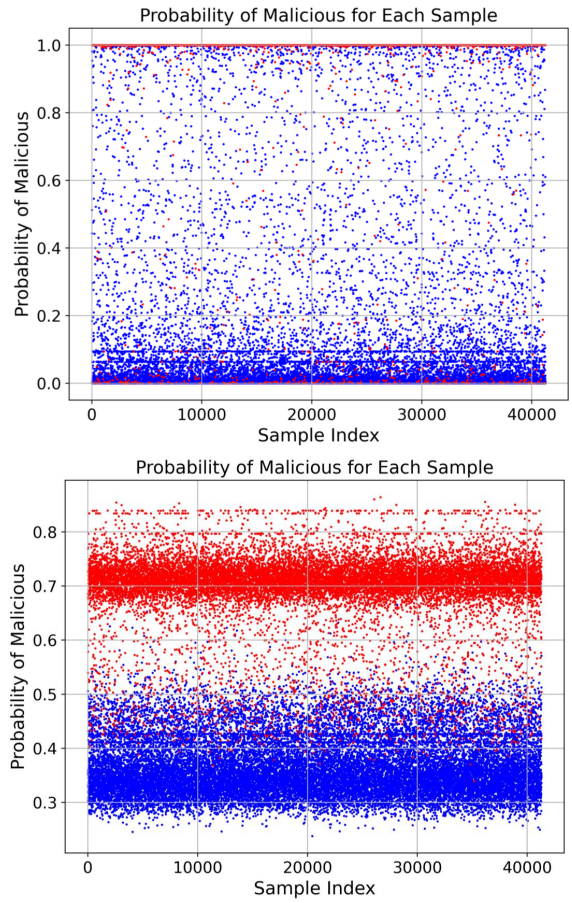


Fig. 6. Illustration of MLP Prediction Value Distribution.

semantic-preserving mutations, these models fail to capture the inherent semantics of SQLi attacks. Their performance on unknown payloads is also poor—LSTM's Recall drops to 67.95% against ML-Driven, and TextCNN's to 59.03% against GPTFuzzer.

Following baseline validation, Naxsi and WAF-Brain will be referenced less frequently in subsequent experiments due to subpar default performance. Similarly, ML-Driven and GPTFuzzer will be mentioned less often: ML-Driven barely bypasses rule-based firewalls and CLCNN, while GPTFuzzer generates excessive redundant data without guaranteeing malicious functionality.

Answer to RQ1: Search-based tools can indeed select more efficiently from the payload set, allowing for rapid bypassing of rule-based firewalls. Mutation-based tools can indeed effectively alter the features of payloads, enabling previously blocked payloads to bypass machine learning-based WAFs. In contrast, generation-based tools require further development to improve their effectiveness and reduce redundancy.

2) Raw Proposed Model Performance: We employed the CLCNN model for feature extraction with a character length of $s = 200$, an embedding layer dimension of $d = 32$, and five sets of filters with kernel sizes $[1, 3, 5, 7, 9]$, each containing 64 randomly initialized convolutional kernels. All experimental code, including parameter configurations and training/test scripts, is available at github repository for reproducibility.[8] We set the parameter λ of the additional loss term in (13) to 0.3, resulting in the changes in confidence distribution shown in Fig. 6. In both figures, the x-axis denotes sample index and the y-axis indicates confidence in predicting maliciousness (blue = benign samples, red = malicious samples). The top figure shows the prediction distribution without the loss term: the model tends to output extreme values, with many misclassified malicious samples clustering near y=0, making it hard to distinguish them from benign samples. After adding the penalty term (bottom figure), the interval [0.4, 0.6] can be defined as uncertain—most misclassified samples concentrate here, while nearly no malicious samples fall below 0.4.

Building upon this, the RL module comprises a two-layer RNN (input dimension matching CLCNN filter count $n_f = 5$, hidden size 16) and a fully connected layer with output dimension equal to the action space size. Table II reports the test-set performance of our CLCNN and hybrid models. With comparable parameter settings, our model achieves accuracy on par with baseline models while yielding higher Recall. Recall variations in Table III further demonstrate our CLCNN model's robustness against adversarial SQLi: even with WAF-A-MoLE, our CLCNN only suffers a 6.94% Recall drop after 10 mutations. Notably, the RL module plays no substantial role here, as it is trained exclusively on the standard training set to prevent incorrect action selections from degrading model performance.

To further validate our model's superiority, we conducted comprehensive adversarial attack tests (see Fig. 5). The left panel shows model Recall changes and convergence trends with increasing mutation counts, covering black-box and gray-box results for WAF-A-MoLE and AdvSQLi. Across all scenarios, CLCNN outperforms baselines with higher Recall and slower declines, demonstrating inherent resilience to mutation-based attacks even without adversarial training. By contrast, word-level feature-based models (LSTM, TextCNN) see Recall drop to as low as 40%, meaning 60% of test-set malicious payloads bypass them within 30 mutations. Nevertheless, CLCNN remains vulnerable to highly obfuscated adversarial payloads. The right panel illustrates the attack success rate variations of search-based tools, with distinct x-axis definitions and uniform success rate calculation rules as follows: For ART4SQLi, the x-axis denotes the number of target bypass payloads to be found; its success rate is calculated as the ratio of the number of target bypass payloads to the total number of attempts. For RAT, which is tasked with finding 100 bypass payloads in total, the x-axis represents the number of attempts per cluster, a design that reflects the distribution characteristics of target detection model vulnerabilities (whether such vulnerabilities are clustered). The baseline of the shaded area in the figure corresponds to 1-Recall. Peaks in the shaded area thus indicate increased bypass success rates relative to random selection, and larger positive shaded areas signify stronger bypass capabilities against the target WAF. It can be observed that neither ART4SQLi nor RAT achieves

[8]https://github.com/lkdxqe/Adversarial-SQLi-Detection

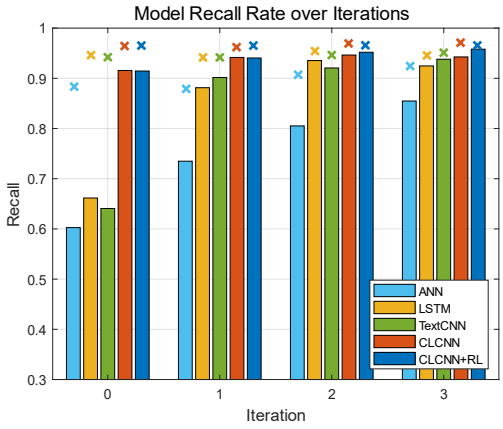


Fig. 7. Illustration of the Variation of Models' 'Recall' with Iterations. Each bar represents the 'Recall' of the model on AdvSQLi(R) with a maximum mutation count of 10 and up to 10 attempts. Each cross in the figure corresponds to the Recall of the respective model on the test set.

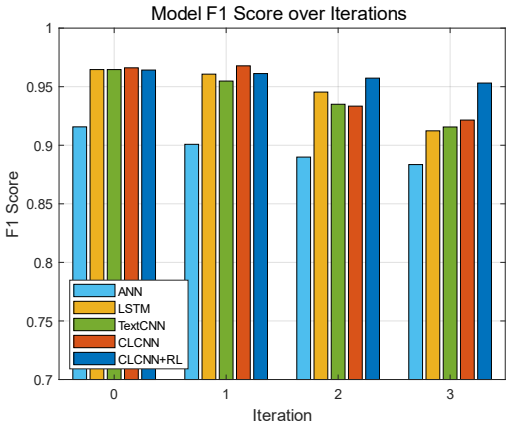


Fig. 8. Illustration of the Variation of Models' F1 Score with Iterations.

significant success rate improvements against CLCNN. For RAT in particular, when the number of attempts per cluster exceeds 20, the shaded area becomes negative, indicating that its bypass performance against our model is worse than random selection.

Answer to RQ2: The CLCNN model achieves high Recall and effectively defends against search-based attacks. However, it struggles to detect highly obfuscated payloads, a limitation that cannot be addressed without incorporating adversarial attack-related knowledge.

C. Model Performance after Adversarial Training

We adopted the random mutation method AdvSQLi(R) to generate adversarial training datasets for each model, aiming to enable models to learn from a broader mutation space and enhance generalization ability. AdvSQLi(R) performs continuous random mutations on each payload in the test set, with up to 10 attempts and a maximum of 10 iterations per attempt, amounting to 100 total possible mutations. The first mutated payload that successfully bypasses the target WAF is added to the adversarial dataset. For baseline models, we oversampled 10,000 entries from their respective adversarial datasets, incorporated them into the training set, and then retrained the models. The hybrid model used the same training set construction approach; however, during adversarial training, only the RL module was trained while the CLCNN module remained fixed. Multi-round adversarial training results are presented in Fig. 7 and Fig. 8. The former depicts the model's Recall changes across training iterations, with AdvSQLi(R) configured for 10 maximum mutations and 10 attempts (the crosses denote test-set Recall in each iteration). The latter shows F1 score variations on the standard dataset. It can be observed that:

1) As the iterations of adversarial training progress, the 'Recall' of all models gradually increase, at the cost of a gradual decline in their performance on the standard dataset.

TABLE IV
MODEL PERFORMANCE UNDER EACH ADVERSARIAL TOOL AS THE ITERATION PROGRESSES.

Detection Methods	Adversarial Attack	Recall(%) in ecah Iteration Round			
		0	1	2	3
ANN	WAF-A-MoLE	78.65	80.44	85.64	87.23
	AdvSQLi(R)	75.78	82.31	87.22	91.94
	ART4SQLi	81.62	82.04	84.91	87.75
	RAT	71.88	70.94	74.92	81.42
LSTM	WAF-A-MoLE	80.06	86.59	92.86	92.21
	AdvSQLi(R)	84.41	92.31	95.16	94.04
	ART4SQLi	90.02	89.89	93.03	91.86
	RAT	83.25	80.67	84.19	83.58
TextCNN	WAF-A-MoLE	75.76	89.27	89.29	92.85
	AdvSQLi(R)	84.26	92.99	94.13	94.99
	ART4SQLi	89.09	88.45	89.83	92.35
	RAT	82.31	82.13	81.27	85.13
CLCNN	WAF-A-MoLE	90.17	93.28	93.65	93.95
	AdvSQLi(R)	94.32	94.89	95.08	95.85
	ART4SQLi	93.80	93.99	94.76	94.91
	RAT	96.16	97.09	97.31	97.31
CLCNN+RL	WAF-A-MoLE	91.57	93.66	93.74	94.20
	AdvSQLi(R)	94.39	94.87	95.80	96.07
	ART4SQLi	93.33	95.04	95.30	95.33
	RAT	96.15	97.36	97.72	97.96

2) After the first round of adversarial training, the resistance of each baseline model to AdvSQLi(R) significantly improves. However, starting from the second round of adversarial training, the improvements become minimal, and there is a noticeable loss in the F1 score.
3) Compared to other models, the 'Recall' of the hybrid model increases relatively slowly, yet it experiences the least decline in performance on the standard dataset, and it is the model for which the 'Recall' is closest to the actual Recall, indicating that AdvSQLi(R) is nearly unable to bypass it within 10 mutations.

From the observations above, we can preliminarily conclude that adversarial training effectively enhances machine learning models' defense against targeted attacks. However, multi-round adversarial training fails to enable them to fully overcome such attacks and often leads to a significant drop in F1 scores on standard datasets. In contrast, our hybrid

TABLE V
THE TASK ALLOCATION ASSESSMENT ON RAW HYBRID MODEL.

Adversarial Tools	Total Acc	CLCNN		RL		
		Ratio	Acc	Ratio	Acc	Acc_noRL
TestSet	**97.20**	0.81	99.85	0.19	85.80	86.67
WAF-A-MoLE	99.08	0.79	99.97	0.21	95.54	92.78
AdvSQLi(R)	99.50	0.96	99.98	0.04	87.91	83.06
AdvSQLi	99.55	0.96	99.97	0.04	89.62	80.67
ART4SQLi	93.02	0.80	99.54	0.20	67.09	66.47
RAT	96.14	0.91	99.55	0.09	59.70	58.99

TABLE VI
THE TASK ALLOCATION ASSESSMENT AFTER ADVERSARIAL TRAINING.

Adversarial Tools	Total Acc	CLCNN		RL		
		Ratio	Acc	Ratio	Acc	Acc_noRL
TestSet	**95.98**	0.81	99.85	0.19	79.41	86.67
WAF-A-MoLE	99.44	0.80	99.94	0.20	97.41	89.82
AdvSQLi(R)	99.73	0.95	99.97	0.05	94.99	82.17
AdvSQLi	99.78	0.95	99.97	0.05	94.12	80.25
ART4SQLi	94.99	0.80	99.58	0.20	79.59	67.19
RAT	97.78	0.92	99.44	0.08	78.92	59.33

model can repeatedly train the RL module to achieve better adversarial resistance at a lower performance cost. Notably, joint training of the CLCNN and RL modules could further improve performance—with the CLCNN focusing on misclassified payloads outside the [0.4, 0.6] confidence interval and the RL targeting cases where the CLCNN initially classifies payloads correctly within [0.4, 0.6] but RL processing leads to misjudgment—but this is not elaborated on in this work. We also evaluated each model's performance against other adversarial tools after training with AdvSQLi(R), as shown in Table IV, with tool parameters consistent with Table III. All models demonstrated enhanced resistance across diverse adversarial tools post-training, with the improvement exceeding the Recall gain on the standard dataset—indicating that strengthened defense is not solely due to higher Recall. Our hybrid model remains the most effective, supporting repeated adversarial training with minimal overfitting risks.

Answer to RQ3: Adversarial training can enhance the model's resistance to different adversarial SQLi tools at the expense of overall performance. But None of these models can completely overcome such attacks. After multiple rounds of adversarial training, our proposed hybrid model approaches its 'Recall' limit when faced with specific adversarial tools, while experiencing a smaller decline in overall performance.

D. The Task Allocation

In this section, we evaluate the respective workload allocation of the CLCNN and RL modules. We first tested the invocation and processing performance of the RL module in the hybrid model against various adversarial tools, with results presented in Tables V and VI. Table V presents the performance of the hybrid model without adversarial training, while Table VI shows the performance after three rounds of adversarial training. For the CLCNN module, $Ratio$ denotes the proportion of payloads classified with sufficient confidence in the initial classification attempt, and Acc represents the accuracy of the CLCNN in identifying these payloads. The RL module is responsible for processing payloads with insufficient confidence: Acc_noRL refers to the classification accuracy of this subset when processed without the RL module (using a threshold of 0.5), and Acc indicates the classification accuracy of this subset after RL module processing. Notably, the accuracy reported in the tables is calculated based on all payloads during the interaction process, thus demonstrating better performance against mutation-based tools than the results discussed previously.

The results indicate that approximately 19% of payloads in the standard dataset require RL module processing. For WAF-A-MoLE and ART4SQLi, 20% of the payloads they mutate or select necessitate RL module handling. In contrast, fewer than 10% of the payloads mutated or selected by AdvSQLi and RAT require RL processing. This suggests that during their strategy adjustment, a considerable number of these payloads are overly simple and thus directly classified by the CLCNN module without RL intervention. Additionally, the CLCNN achieves a minimum accuracy of 99.44% for decisions when confronting RAT configured with 20 attempts per cluster, and generally delivers superior overall performance. While the RL module exhibits subpar performance on the standard test set, with incorrect action selections degrading model performance, it demonstrates improved accuracy against all other adversarial tools. To address this discrepancy, we can adjust the ratio of standard dataset samples to adversarial payloads during RL module training and modify the reward mechanism for the 'skip' action, encouraging the model to accept CLCNN classification results while only activating in response to highly obfuscated payloads.

We also examined the actions that the RL module tends to take after three rounds of adversarial training. As shown in Fig. 9, the x-axis represents the frequency proportions of each action, while the y-axis lists the abbreviations of the various actions in the action space. During the experimental process, we further categorized the actions related to comment handling. Specifically, $RC1$ matches and removes '/* */' along with its enclosed content, $RC2a$ matches the last '#' in the payload and removes everything following it, $RC2b$ matches the last '--' in the payload and removes everything following it, and $RC3$ matches '/*! */' while retaining its content. Since our current strategy is to retain special symbols such as '#' and '--', the model may call the function for removing comments multiple times under the same task, resulting in an artificially high frequency for RC2a and RC2b in Fig. 2.

After imposing limits on repeated actions, the average number of actions executed by the RL module when facing adversarial tools can be reduced to 2.8. This indicates that at most 20% of the payloads will invoke the CLCNN an average of 3.8 times in the cycle shown in Fig. 2 resulting in a maximum average of 1.76 calls to the CLCNN module. To balance model efficiency and accuracy, further adjustments can be made to the selection of uncertainty intervals, the increase

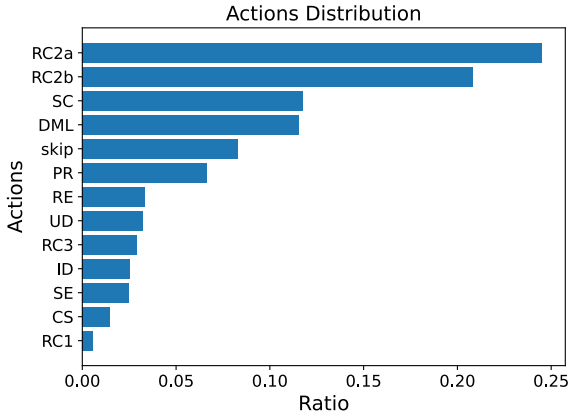


Fig. 9. Illustration of the Action Distribution Output by the RL Module.

of rewards for "skip," and the execution of more operations within a single action.

Answer to RQ4: Approximately 80% of the payloads are classified directly by the CLCNN module, achieving an average accuracy exceeding 99.5%. Among the remaining 20% of payloads, the RL module effectively handles those generated by mutation-based tools, with an average accuracy exceeding 95%.

E. Computational Overhead

While previous sections have validated the superior detection performance of the proposed model, this subsection focuses on evaluating its practical applicability in real-world environments by measuring computational overhead and resource consumption. To simulate realistic web service scenarios, we deployed a web interface using Python Flask on a remote server. The interface receives user requests, feeds them as inputs to the detection models, returns the confidence score of the request being classified as malicious, and records the model's inference time and memory usage using the psutil library.

To ensure the fairness and reliability of the experiments, each detection model was independently encapsulated using Docker, providing separate external interfaces that run concurrently. However, only one interface was invoked at a time during each stress test. We evaluated five machine learning-based models: ANN, CLCNN, CLCNN+RL, LSTM, and TextCNN. Rule-based WAFs (e.g., ModSecurity, Naxsi) were excluded for two reasons: first, these models are typically pre-packaged and not under our direct control, making it impossible to accurately measure their inference time in a remote environment; second, rule-based and machine learning-based models have distinct design priorities and trade-offs, and our goal is to demonstrate the competitiveness of our model among machine learning-based alternatives.

Experimental results are presented in Fig. 10 and Table VII, with all tests conducted on a remote server equipped with an Intel Core i5-13400 CPU (no GPU acceleration) to ensure fair comparison under realistic deployment constraints. Fig. 10 shows the response time of each interface for the first 3000 requests: the solid line represents exact data from one test, while error bands of the same color indicate the time overhead fluctuation across 10 repeated tests. Quantitative analysis yields the following average response times: ANN (0.68 ms), CLCNN (3.84 ms), CLCNN+RL (4.30 ms), TextCNN (43.81 ms), and LSTM (46.60 ms).

TABLE VII
MEMORY CONSUMPTION OF DIFFERENT DETECTION MODELS

Model	Physical Memory (MB)	Virtual Memory (MB)
ANN	113.42	1624.42
CLCNN	261.01	2248.40
CLCNN+RL	322.81	2221.65
LSTM	772.51	7726.58
TextCNN	781.23	7763.23

As observed, the statistics-based ANN model has the lowest response time (consistent with its earlier reported inferior detection performance), followed by CLCNN. The CLCNN+RL hybrid model maintains a baseline response time nearly identical to CLCNN, with its curve closely aligning—its average response time is only 12% higher, reflecting modest overhead for enhanced adversarial detection. However, CLCNN+RL occasionally shows higher peak pulses (rare among 3000 consecutive requests), due to RL module invocation for highly obfuscated payloads requiring multi-round action loops and additional computational overhead. In contrast, LSTM and TextCNN (relying on Word2Vec tokenization) incur significantly higher time overhead, with average response times one level above tokenization-free models.

Table VII illustrates the average memory consumption (including physical memory and virtual memory) of each interface during operation. The resource consumption ranking of the models aligns with their time overhead: tokenization-based models (LSTM, TextCNN) exhibit drastically higher memory usage, driven by the storage demands of pre-trained Word2Vec embeddings and intermediate sequential feature maps. In contrast, the CLCNN and CLCNN+RL models maintain far more efficient resource utilization, with CLCNN+RL only incurring a modest 23.7% increase in physical memory usage compared to the base CLCNN model (322.81 MB vs. 261.01 MB) and nearly identical virtual memory consumption (2221.65 MB vs. 2248.40 MB).

In summary, the real-world deployment feasibility evaluation, conducted via Docker-containerized remote Flask web interfaces, confirms that the CLCNN+RL hybrid model maintains acceptable computational overhead and resource consumption. While its average latency and memory usage are slightly higher than those of CLCNN (a reasonable trade-off for enhanced adversarial SQLi detection capabilities), both metrics are significantly lower than those of tokenization-based LSTM and TextCNN models. The statistics-based ANN model, despite having the lowest latency and memory footprint, lacks effective detection performance against adversarial payloads. Collectively, these results validate CLCNN+RL as a practically viable choice that balances defense performance and deployment efficiency for real-world web service architectures.

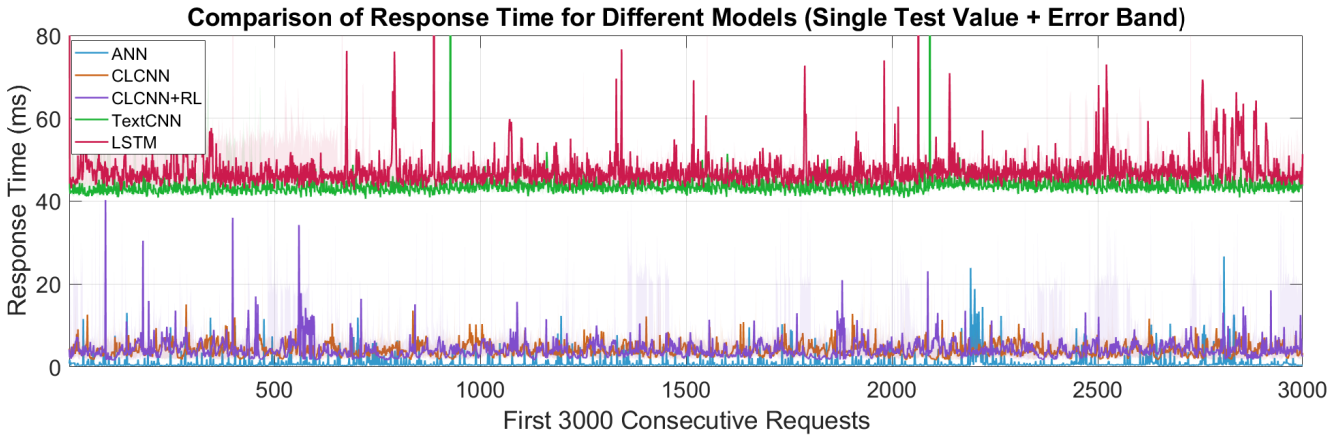


Fig. 10. Response Time of Different Detection Models Under Real-World Deployment Conditions. Each model was tested 10 times with 10,000 consecutive requests sent in each test; the solid lines in different colors represent the response time of the first 3,000 requests in the first test for each model, while the lighter error bands of the same color reflect the fluctuation range of time overhead across all 10 rounds of tests.

Answer to RQ5: The proposed CLCNN+RL model is feasible for real-world deployment. Its computational overhead and resource consumption, though slightly higher than CLCNN, are acceptable and outperform tokenization-based models.

V. Conclusion

This paper validates the significant challenges posed by adversarial SQLi attacks to mainstream detection methods: statistical/word-level ML models are vulnerable to mutation-based attacks, while rule-based methods are easily exploited by search-based tools. Given their accessibility, these adversarial tools exacerbate web security risks, calling for targeted defenses. To address this, we propose a hybrid CLCNN+RL model. The enhanced CLCNN captures malicious patterns through character-level multi-scale feature extraction, exhibits obfuscation resistance, and eliminates dependencies on tokenization. The plug-and-play RL module, trained adversarially, performs targeted de-obfuscation on ambiguous payloads—overcoming overfitting from direct CLCNN retraining while boosting resistance to adversarial attacks. Experimental results confirm the model's superior performance against diverse adversarial SQLi tools. Notably, the framework exhibits strong scalability for other web attack detection tasks. By integrating the RL module with domain-specific base models (using their hidden layer outputs as input, with frozen parameters) and updating the action space via inverting task-specific mutation operators, it enables low-cost migration without compromising core defensive capabilities.

In summary, this work provides an effective solution for adversarial SQLi detection and a scalable framework for broader web adversarial attack defense. Future research will focus on improving the performance of the RL module and validating the framework's effectiveness across more real-world application scenarios.

References

[1] D. Appelt, C. D. Nguyen, A. Panichella, and L. C. Briand, "A machine-learning-driven evolutionary approach for testing web application firewalls," *IEEE Transactions on Reliability*, vol. 67, no. 3, pp. 733–757, 2018.

[2] H. Liang, X. Li, D. Xiao, J. Liu, Y. Zhou, A. Wang, and J. Li, "Generative pre-trained transformer-based reinforcement learning for testing web application firewalls," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 1, pp. 309–324, 2023.

[3] Z. Qu, X. Ling, T. Wang, X. Chen, S. Ji, and C. Wu, "Advsqli: Generating adversarial sql injections against real-world waf-as-a-service," *IEEE Transactions on Information Forensics and Security*, 2024.

[4] L. Zhang, D. Zhang, C. Wang, J. Zhao, and Z. Zhang, "Art4sqli: The art of sql injection vulnerability discovery," *IEEE Transactions on Reliability*, vol. 68, no. 4, pp. 1470–1489, 2019.

[5] M. Amouei, M. Rezvani, and M. Fateh, "Rat: Reinforcement-learning-driven and adaptive testing for vulnerability discovery in web application firewalls," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3371–3386, 2021.

[6] D. Wang, Z. Zhang, P. Wang, J. Yan, and X. Huang, "Targeted online password guessing: An underestimated threat," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 2016, pp. 1242–1254.

[7] Q. Wang, D. Wang, C. Cheng, and D. He, "Quantum2fa: Efficient quantum-resistant two-factor authentication scheme for mobile devices," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 1, pp. 193–208, 2021.

[8] S. Roy, A. K. Das, S. Chatterjee, N. Kumar, S. Chattopadhyay, and J. J. Rodrigues, "Provably secure fine-grained data access control over multiple cloud servers in mobile cloud computing based healthcare applications," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 1, pp. 457–468, 2018.

[9] M. Xu and D. Wang, "Practical two-factor authentication protocol for real-time data access in wsns," *IEEE Transactions on Dependable and Secure Computing*, 2025.

[10] D. Appelt, C. D. Nguyen, L. C. Briand, and N. Alshahwan, "Automated testing for sql injection vulnerabilities: an input mutation approach," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 2014, pp. 259–269.

[11] L. Demetrio, A. Valenza, G. Costa, and G. Lagorio, "Waf-a-mole: evading web application firewalls through adversarial machine learning," in *Proceedings of the 35th Annual ACM Symposium on Applied Computing*, 2020, pp. 1745–1752.

[12] M. Hemmati and M. A. Hadavi, "Using deep reinforcement learning to evade web application firewalls," in *2021 18th International ISC Conference on Information Security and Cryptology (ISCISC)*. IEEE, 2021, pp. 35–41.

[13] T. Y. Chen, F.-C. Kuo, R. G. Merkel, and T. Tse, "Adaptive random testing: The art of test case diversity," *Journal of Systems and Software*, vol. 83, no. 1, pp. 60–66, 2010.

[14] D. Appelt, C. D. Nguyen, and L. Briand, "Behind an application firewall, are we safe from sql injection attacks?" in *2015 IEEE 8th international conference on software testing, verification and validation (ICST)*. IEEE, 2015, pp. 1–10.

[15] K. Papineni, "Why inverse document frequency?" in *Second meeting of the North American chapter of the association for computational linguistics*, 2001.

[16] D. E. Denning, "An intrusion-detection model," *IEEE Transactions on software engineering*, no. 2, pp. 222–232, 1987.

[17] A. Perez-Villegas and G. Alvarez, "An anomaly-based web application firewall," in *International Conference on Security and Cryptography*, vol. 1. SCITEPRESS, 2009, pp. 23–28.

[18] P. Tang, W. Qiu, Z. Huang, H. Lian, and G. Liu, "Detection of sql injection based on artificial neural network," *Knowledge-Based Systems*, vol. 190, p. 105528, 2020.

[19] S. Ramezany, R. Setthawong, and T. Tanprasert, "A machine learning-based malicious payload detection and classification framework for new web attacks," in *2022 19th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON)*. IEEE, 2022, pp. 1–4.

[20] L. Zhou, T. Lu, and X. Hu, "Detecting web application injection attacks using one-class svm," in *2022 IEEE 5th International Conference on Computer and Communication Engineering Technology (CCET)*. IEEE, 2022, pp. 275–279.

[21] L. Yu, L. Chen, J. Dong, M. Li, L. Liu, B. Zhao, and C. Zhang, "Detecting malicious web requests using an enhanced textcnn," in *2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2020, pp. 768–777.

[22] H. Kim and Y. Yoon, "An ensemble of text convolutional neural networks and multi-head attention layers for classifying threats in network packets," *Electronics*, vol. 12, no. 20, p. 4253, 2023.

[23] Q. Li, F. Wang, J. Wang, and W. Li, "Lstm-based sql injection detection method for intelligent transportation system," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 5, pp. 4182–4191, 2019.

[24] J. R. Tadhani, V. Vekariya, V. Sorathiya, S. Alshathri, and W. El-Shafai, "Securing web applications against xss and sqli attacks using a novel deep learning approach," *Scientific Reports*, vol. 14, no. 1, p. 1803, 2024.

[25] B. G. Bokolo, L. Chen, and Q. Liu, "Detection of web-attack using distilbert, rnn, and lstm," in *2023 11th International Symposium on Digital Forensics and Security (ISDFS)*. IEEE, 2023, pp. 1–6.

[26] S. Salim and O. Lahcen, "A bert-enhanced exploration of web and mobile request safety through advanced nlp models and hybrid architectures," *IEEE Access*, 2024.

[27] R. Tang, Z. Yang, Z. Li, W. Meng, H. Wang, Q. Li, Y. Sun, D. Pei, T. Wei, Y. Xu *et al.*, "Zerowall: Detecting zero-day web attacks through encoder-decoder recurrent neural networks," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 2479–2488.

[28] M. Schwarzl, P. Borrello, G. Saileshwar, H. Müller, M. Schwarz, and D. Gruss, "Practical timing side-channel attacks on memory compression," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 1186–1203.

[29] S. Das, T. Yurek, Z. Xiang, A. Miller, L. Kokoris-Kogias, and L. Ren, "Practical asynchronous distributed key generation," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 2518–2534.

[30] J. Saxe and K. Berlin, "expose: A character-level convolutional neural network with embeddings for detecting malicious urls, file paths and registry keys," *arXiv preprint arXiv:1702.08568*, 2017.

[31] M. Ito and H. Iyatomi, "Web application firewall using character-level convolutional neural network," in *2018 IEEE 14th International Colloquium on Signal Processing & Its Applications (CSPA)*. IEEE, 2018, pp. 103–106.

[32] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "Cbam: Convolutional block attention module," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 3–19.

[33] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial intelligence*, vol. 101, no. 1-2, pp. 99–134, 1998.

[34] X. Wang, S. Wang, X. Liang, D. Zhao, J. Huang, X. Xu, B. Dai, and Q. Miao, "Deep reinforcement learning: A survey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 4, pp. 5064–5078, 2022.

[35] R. S. Sutton, "Learning to predict by the methods of temporal differences," *Machine learning*, vol. 3, pp. 9–44, 1988.

[36] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, no. 1, 2016.

[37] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.

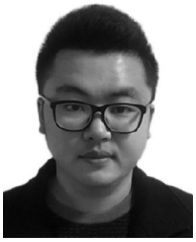
Fucai Yu received his B.S. degree from Lanzhou Railway college in 1999, majoring in communication engineering, and received his master and PH.D. degrees in computer communication and security from Chungnam National University, Korea, in 2006 and 2010 respectively. Currently, he is an associate professor in School of Information and Communication Engineering, University of Electronic Science and Technology of China (UESTC), P.R. China. His research interests include network security, social network analysis, anonymous communication and network traffic classification.

Xusheng Li received his B.S. degree from the School of Information and Communication Engineering, University of Electronic Science and Technology of China (UESTC), P.R. China. He is currently studying for the master degree in College of Information and Communication Engineering, UESTC. His research interests include network security and artificial intelligence technology.

Yang Wu received his B.S.degrce from University of Electronic Science and Technology of China (UESTC) in 2017, majoring in Earth Information Science and Technology, and received his master degree in Information and Communication Engineering from UESTC in 2024. He is now an engineer of China Academy of Launch Vehicle Technology. His research interests include network security, anonymous communication, etc.

Ziqiang Chang received his B.S. degree from Sichuan University in 2020, majoring in Electronic Information Engineering, and received his master degree in School of Information and Communication Engineering from University of Electronic Science and Technology of China (UESTC) in 2023. He is currently an engineer in Luoyang Institute of Electro-Optical Equipment. His research interests include network security, network analysis, Image recognition.

Gaolei Fei received his B.S. and Ph.D. degrees from the School of Information and Communication Engineering, University of Electronic Science and Technology of China (UESTC), Sichuan Province, P.R. China in 2006 and 2012, respectively. From 2010 to 2011, he was a graduate research trainee at McGill University, Montreal, QC, Canada. He is now a Full Professor at the School of Information and Communication Engineering, UESTC, P.R. China. His current research interests include network topology measurement and online social network analysis.



Xuemeng Zhai received the bachelor's and PhD degrees from the University of Electronic Science and technology of China (UESTC) in 2014 and 2020, respectively. He is currently an associate research fellow with the School of Information and Communication Engineering, UESTC, China. His research interests include network science, complex network, community detection, and network sparse representation.